CVML Assignment 4

Moved Object Detection Using DETR

CSCI-3033.076 *Vision Meets Machine Learning* — Spring 2026

What Assignments 3 gave you. In Assignment 3, you built a data-loading pipeline, implemented bipartite matching using the Hungarian algorithm, and evaluated matching quality on VIRAT frame pairs. **Assignment 4** takes this further: instead of hand-crafted cost functions, you will fine-tune a DETR model to *learn* to detect objects that have moved between two frames.

Overview

In this assignment you will build an end-to-end pipeline to detect objects that have moved between two frames of surveillance footage. You will fine-tune a pre-trained DETR model (`facebook/detr-resnet-50`) from HuggingFace Transformers to perform this task on the VIRAT dataset.

You will:

1. Use the provided `data_ground_truth_labeller.py` script to generate ground truth annotations for moved objects.
2. Compute pixel-wise image differences between frame pairs and use them as input to DETR.
3. Fine-tune DETR under multiple fine-tuning strategies and compare their performance.
4. Evaluate your model with precision, recall, and qualitative visualizations.
5. Write a technical report documenting the entire pipeline from data to results.

Learning Objectives

By the end of this assignment you will be able to:

1. Understand how DETR combines a CNN backbone with a transformer for moving object detection.

Input Data

- **Source:** VIRAT surveillance dataset, processed frame pairs are provided to you as a .sqf file. You can find it here: /scratch/VML26SP/frame_pair_data.sqf
- **Structure:** Frame pairs with their respective `.annotation.txt` files from surveillance camera footage.
- **Object classes:** Unknown=0, Person=1, Car=2, Other Vehicle=3, Other Object=4, Bike=5.
- **Annotation format:** Same 8-column format as in Assignment 3 Part 2 pdf doc.

Read more about the VIRAT dataset here:
`https://data.kitware.com/#item/56f578dc8d777f753209bdef`

Data Ground Truth Labeling

Before training, you need to generate annotations that indicate which objects *moved* between two frames. The raw per-frame annotations only tell you where objects are in each frame individually; they do not tell you which objects changed position across frames.

Why is this step needed? Your per-frame annotation files describe bounding boxes independently for each frame. To train a model that detects *moved* objects, you need ground truth that (1) matches objects across frames and (2) keeps only the ones that actually changed position. That is what the labeller script produces.

Using the Labeller Script

Use the provided `data_ground_truth_labeller.py` to generate matched annotations. The script does the following:

1. **Feature extraction:** Crops each detected object from the frame and extracts a feature vector using a pre-trained ResNet-18.
2. **Cost matrix construction:** Builds a cost matrix combining cosine distance between features, normalized centroid distance, object type penalty, and bounding box size penalty.
3. **Hungarian matching:** Uses `scipy.optimize.linear_sum_assignment` to find the optimal one-to-one assignment between objects in Frame 1 and Frame 2.
4. **Movement filtering:** Keeps only matched pairs where the IoU between the old and new bounding box is below a threshold (default 0.1), meaning the object moved significantly.

Output format. Each output annotation file contains 2 rows per moved object. The first row is the old bounding box (from Frame 1) and the second row is the new bounding box (from Frame 2). The format of each row is:

```
<object_id> <x> <y> <w> <h> <object_type>
```

Modifications allowed. You may modify the labeller script (e.g., change the IoU threshold, adjust cost weights, add augmentation). However, if you do so, you **must** document and explain every change in your report.

Model Architecture: Pixel-Difference Input to DETR

The core idea is simple: compute the pixel-wise difference between the two frames and feed that difference image as input to the DETR model. The difference image highlights regions where objects have moved, and DETR learns to detect bounding boxes around those regions.

Pipeline

1. **Preprocessing:** Given a frame pair (Frame 1, Frame 2), resize both frames to the same spatial dimensions if they differ. Then compute:

$$I_{\text{diff}} = |I_{\text{frame2}} - I_{\text{frame1}}|$$

where $|\cdot|$ denotes the element-wise absolute value. I_{diff} is a 3-channel image of the same size as the input frames.

2. **Input to DETR:** Feed I_{diff} as the input image to the full DETR model (`facebook/detr-resnet-50`). The difference image passes through the ResNet-50 backbone, then through the transformer encoder-decoder, and finally through the detection heads.

3. **Target:** The ground truth bounding boxes are the ***new*** **positions** (Frame 2 coordinates) of the moved objects, as produced by the labeller script.

Why pixel difference? Subtracting two frames suppresses the static background and amplifies regions where objects have appeared, disappeared, or shifted. This is a classical technique in video surveillance (background subtraction) and gives the model a strong prior about where to look for changes.

Image size handling. Make sure both frames are resized to matching dimensions before computing the difference. Think about what interpolation method is appropriate and whether you need to adjust the bounding box coordinates accordingly. Document your choice in the report.

Fine-Tuning Strategies

You must provide comparison studies across the following fine-tuning strategies. Run a full training loop for each and report results.

1. **Fine-tune only the convolutional backbone.** Freeze the transformer and detection heads. Only update the ResNet-50 backbone parameters.

2. **Fine-tune only the transformer classification head.** Freeze both the backbone and the transformer encoder-decoder. Only update the final fully-connected layers used for class prediction and bounding box regression.

3. **Fine-tune only the transformer block.** Freeze the backbone and the classification head. Only update the transformer encoder and decoder parameters.

Design justification required. For each strategy, explain in your report *why* you would expect it to perform well or poorly on this task. Consider factors like dataset size, domain gap from COCO (DETR's pre-training data), and which components of the model need to adapt most.

Training and Evaluation

Data Split

Use an 80/20 train-test split on the matched annotation dataset. You may create a validation set from the training split (e.g., 80/10/10 or similar) for hyperparameter tuning and early stopping.

Evaluation Metrics

You must report the following for each fine-tuning strategy:

- **Precision:** Of all predicted bounding boxes, what fraction correctly correspond to a moved object?
- **Recall:** Of all ground truth moved objects, what fraction did the model successfully detect?

You may additionally report mAP (mean Average Precision), F1 score, or any other metric you find informative.

Qualitative Evaluation

Include visualizations of model predictions on test samples. Show the predicted bounding boxes overlaid on the difference image and/or the original Frame 2. Use different colors for different object classes.

Submission Checklist

Report

Submit a technical report (PDF, strictly should be less than 4 pages) containing:

a. **Pipeline description:** How you generated the ground truth, how you computed the pixel difference, and any preprocessing steps.

b. **Architecture details:** Which DETR variant you used, how you set up the input pipeline, and the number of object queries used.

c. **Fine-tuning comparison:** A table or set of plots comparing precision and recall across all four fine-tuning strategies. Include training curves (loss vs. epoch) for each strategy.

d. **Qualitative results:** At least 5 visualization images showing model predictions on diverse test pairs.

e. **Design justification:** Rationale behind your choices (image size, learning rate, number of epochs, batch size, etc.).

f. **Assumptions and limitations:** Any edge cases, failure modes, or workarounds you encountered.

Code

Submit a zip file containing your program files. The code should be modular:

- Separate files for the dataloader, model setup, trainer, configuration, and evaluation.
- A `README.md` with instructions on how to run the code.
- SLURM batch scripts used for training on the HPC cluster.
- SLURM output logs and screenshots demonstrating that the code was run by you.

Do NOT include data files or model weight tensors in your submission. Only submit code, scripts, logs, and the report.

Recommended Libraries

Library	Purpose
`transformers` (HuggingFace)	Pre-trained DETR model and processor
`torch` / `torchvision`	Training loop, transforms, data loading
`opencv-python` (`cv2`)	Image I/O, resizing, pixel difference
`numpy`	Array operations
`matplotlib`	Visualization of predictions and training curves
`scipy`	Hungarian matching (used inside the labeller)

Grading Rubric

Component	Points
Ground truth generation (correct use of labeller, documentation of any changes)	10
Pixel-difference pipeline (correct preprocessing, resizing, difference computation)	15
DETR fine-tuning setup (correct model loading, loss function, training loop)	15
Fine-tuning comparison (all 3 strategies implemented and compared)	20
Evaluation metrics (precision, recall reported correctly for each strategy)	10
Qualitative visualizations (at least 5 diverse, well-annotated examples)	10
Report quality (clear writing, justified design choices, training curves)	10
Code quality (modular structure, README, SLURM scripts and logs)	10
Total	**100**

Suggested Project Structure

```
{netid}_{name}_CV_4/
  data_ground_truth_labeller.py   # provided (or modified) labeller
  dataset.py                      # custom Dataset class
  model.py                        # DETR setup and freezing logic
  train.py                        # training loop
  evaluate.py                     # evaluation metrics and visualization
  config.py                       # hyperparameters and paths
  run_train.sbatch                # SLURM batch script
  slurm_outputs/                  # SLURM log files
  README.md                       # how to run
  report.pdf                      # technical report
```